

程序设计语言初步

-算术运算

软件与理论中心 张艳梅



- ◆ 典型的运算表达式有以下四种：
 1. 算术运算符 (+, -, *, /, %) 和算术表达式；
支持浮点数运算、整数运算、整数除法、取余运算
 2. 赋值运算符 (=, +=, -=, *=, /=, %=) 和赋值表达式；
例：`result = a+b;`
 3. 关系运算符(>, <, ==, >=, <=, !=) 和关系表达式；
 4. 逻辑运算符(!, &&, ||) 和逻辑表达式；

赋值表达式是带有赋值符=的表达式;

当变量基于自身数值更新运算时，C语言很贴心地定义了简写形式：

`num += 5;` `/*是num = num+5;的简写*/`

`num -= 5;` `/*是num = num-5;的简写*/`

`num *= 5;` `/*是num = num*5;的简写*/`

`num /= 5;` `/*是num = num/5;的简写*/`

`num %= 5;` `/*是num = num%5;的简写*/`

适当加入注释

在 C 语言中，单行注释从 “//” 开始直到行末为止；多行注释用 “/*” 和 “*/” 包围起来。

提示：适当在程序中编写注释不仅能让其他用户更快地搞懂你的程序，还能帮你自己理清思路。

◆ 算术运算符有以下五种：

1. +加法

2. -减法

3. *乘法

4. 除法,两个整数相除按整数除法规则,至少一个浮点数参与的除法是实数除法。

例: $3/5$, $2.48/1.03$, $1/3.14$ 。

5. % 取余, 整数除法算法中的余数计算。

算术运算的优先级

- 编程语言规定了运算符的优先级。

在表达式求值时, 先按运算符的优先级别高低次序执行, 例如先乘除后加减。

例: $17*5+8/3-15$

Operator	Name
!	Logical NOT. Bang.
++ --	Increment and decrement operators.
* / %	Multiplicative operators.
+ -	Additive operators.
<< >>	Shift operators.
< > <= >=	Inequality comparators.
== !=	Equality comparators
&	Bitwise AND.
^	Bitwise XOR.
	Bitwise OR.
&&	Logical AND.
	Logical OR.
?:	Conditional.
= op=	Assignment.

- 编程语言规定了各种运算符的结合方向(结合性)
C语言中默认的算术运算符的结合方向为“自左至右”,即先左后右。
但是赋值运算符是右结合,即从右到左计算。
例: `a=b=c;` //先将c的值赋给b,再将b的值赋给a。
- ANSIC并没有具体规定表达式中的子表达式的求值顺序,允许各编译系统自己安排。

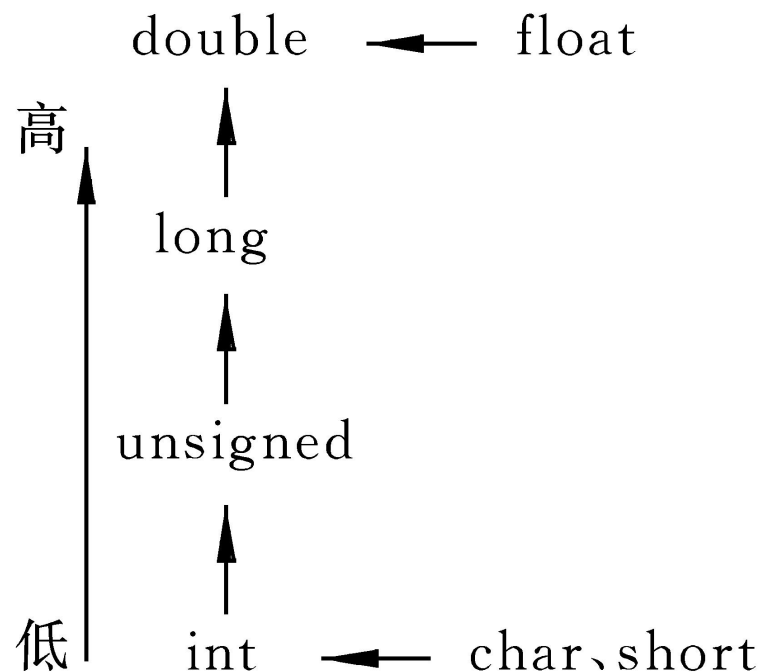
```
/* rules.c --优先级测试 */  
#include <stdio.h>  
int main(void)  
{  
    int top, score;  
    top = score = -(2+5)*6 + (4+3*(2+3));  
    printf("top=%d,score=%d\n",top,score);  
    return 0;  
}
```


各类型数据的混合运算

混合运算：整型(包括int, short, long)、浮点型(包括float, double)可以混合运算。在进行运算时, 不同类型的数据要先转换成同一类型, 然后进行运算.

说明:

这种类型转换是由系统自动进行的。



- 请用C语言的运算表达式写出下列问题计算式。

- 三角形面积公式（1）：底乘高的一半

- 三角形面积公式（2）：海伦公式

$$S = \sqrt{p(p-a)(p-b)(p-c)}$$

- 求直角三角形斜边长

- 一元二次方程求根公式，有实根的情况

- 语法:

```
#include <math.h>
```

```
double sqrt( double num );
```

功能: sqrt函数返回参数num的平方根。如果num为负,产生域错误。例:

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main(void){
```

```
    double a=9.0;
```

```
    printf("sqrt(9) is %lf.\n",sqrt(a));
```

```
    printf("sqrt(2) is %.8lf\n",sqrt(2.0));
```

```
    printf("sqrt(9/4) is %.8lf\n",sqrt(a/4.0));
```

```
}
```

可以利用强制类型转换运算符将一个表达式转换成所需类型。

格式: **(类型名)** (表达式)

例: `/* datatypeTrans.c */`

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int main(void){
```

```
    int a=3;
```

```
    float b=3.1415;
```

```
    double c=8.9e-7;
```

```
    printf("int %d to float %f.\n",a,(float)a);
```

```
    printf("float %f to double %lf.\n",b,(double)b);
```

```
    printf("double %lf to int %d.\n",c,(int)c);
```

```
} //思考: 强制类型转换, 和类型不匹配的结果为何不同?
```

我们先从熟悉的算术运算入手，看看如何用计算机编程完成各类算术运算。

1. 定义程序目标

问题：制作一个计算器，输入数据a和b，程序应输出 $a+b$ 的结果。

说明：明确程序需要哪些信息，进行什么计算，报告什么结果。

2. 设计程序

问题：程序用户是谁？

问题：有多久的开发时间？

问题：程序界面如何？

最简单的字符界面和键盘交互。

问题：如何表示数据？

整数，还是实数（精度）？

问题：如何处理数据？

a+b计算器的算法设计：

Begin

定义两个实数变量a和b；

从键盘接受输入的两个实数，存入a和b；

定义实数变量result；

计算a+b将结果存入result；

显示器输出打印result的值；

End

3. 编写代码

首先：选择编程语言。

然后：根据算法，对应所选编程语言的编码规范；

算法需要数据变量表示和算术运算，以及输入、输出操作。

通过预先定义的输入函数可以从输入设备接收信息。

C语言的格式化输入函数：

scanf(格式定义串[,参数]...);

功能：从标准输入设备（键盘）上读取一系列数据，按格式定义串的要求进行转换并送到参数表所列的变量中。

`scanf("%d",&x);`//读取一个整数（%d），按地址存入变量x

`scanf("%d%d",&x,&y);`//读取两个整数，按地址存入变量x和y

`scanf("%f%f",&a,&b);`//读取两个实数，按地址存入变量a和b

`scanf("%lf",&d);`//读取一个双精度实数，按地址存入变量d

通过预先定义的输出函数可以将值传送到输出设备。

C语言的格式化输出函数：

printf(格式定义串[,参数]...);

功能：将参数列表的变量/常量或表达式值，按格式定义显示在标准输出设备（一般为显示器）上。

`printf("x=%d\n",x);`//打印x=整数

`printf("%d%d",a,b);`//打印a和b两个整数，**紧挨在一起**

`printf("%f%f",a,b);`//打印a和b两个实数，中间有空格

`printf("%.2f%.2f",a,b);`//打印a和b两个实数，**只显示两位小数**

`printf("%.16f\n",d);`//打印d双精度实数，**显示16位小数**

3. 编写代码

首先：选择编程语言。

然后：根据算法，对应所选编程语言的编码规范；

最后：编写代码。

建议：打开 C 开发环境，新建一个文件，编辑代码，然后保存文件，注意设定存储路径和选择c类型后缀。点击菜单的Compile编译，如果有编译错误，改正。编译成功后，点击Run运行。

/*程序包含预先定义的常用函数*/

#include <stdio.h>

#include <stdlib.h>

/*C程序的固定主函数main(), 以{}为界。*/

int main(void){

/*程序员自定义的内容。*/

printf("Hello world!\n");

/*注意所有标点符号必须是英文符号*/

/*给出函数返回值, 对main函数来说是固定写法*/

return 0;

}

注: 如果程序运行失败"*Failedtoexecute*.exe*",请把杀毒软件关掉后再试。

编程活动步骤3-编写代码

a+b计算器的算法设计:

Begin

定义两个实数变量a和b;

从键盘接受输入的两个
实数, 存入a和b;

定义实数变量result;

计算a+b将结果存入

result;

显示器输出打印result
的值;

End

```
/*在程序员自定义内容区域敲代码  
*/
```

```
float a,b,result;  
printf("Please input two  
real numbers:\n");  
scanf("%f%f",&a,&b);  
result=a+b;  
printf("%f+%f=%f\n",a,b,res  
ult);
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

如果有错误，查看错误报告，理解错误信息；然后修正代码后重新编译。

使用IDE辅助功能：关键字会加粗显示，常量、运算符、语法符号(语句结束符、括号)会彩色显示，注释会灰掉。

5. 测试和调试程序

原因：自己很容易犯错误。

用测试数据运行程序，查看结果是否有错。

测试数据1: 2.3, 4.9;

测试数据2: 0, 3;

测试数据3: -3, 3;

测试数据4: 1234.78, 9876.54;

如果有错误，分析错误原因，测试确认错误原因；
然后修正代码后重新编译。

5. 测试和调试程序

思考：设计的算法在什么情况下会出错？

当a和b足够大， $a+b$ 的结果会超出计算机表示范围吗？

测试数据5: 1234567890.1, 9876543210.1;

防止溢出：把float改为double类型

```
/*在程序员自定义内容区域敲代码*/  
double a,b,result;  
printf("Please input two real numbers:\n");  
scanf("%lf%lf",&a,&b);  
result=a+b;  
printf("%.16f+%.16f=%.16f\n",a,b,result);
```

思考：数据溢出可以完全避免吗？如果不能杜绝溢出，应该如何合理地避免溢出？

问题：圆柱体的表面积

输入 底面 半径 r 和 高 h ，输出 圆柱体 的 表面积，**保留 3 位 小数**，格式 见 样例。

样例 输入： 3.5 9

样例 输出： $\text{Area} = 274.889$

问题摘自刘汝佳. 算法竞赛入门经典 (第2版)

求圆柱体表面积的计算设计

求圆柱体表面积的计算设计：

Begin

定义浮点数变量radius, height, area;

从键盘接受输入的半径和高度，存入radius, height;

计算圆柱体的两个底面积和侧面积，然后求和；

$$\begin{aligned} \text{area} = & (\pi * \text{radius} * \text{radius}) * 2 \\ & + (2 * \pi * \text{radius} * \text{height}) \end{aligned}$$

显示器输出打印area, 保留3位小数；

End

问题：应该选择单精度还是双精度类型？

π 取多少位合适？

两处的 π 应该保持一致.

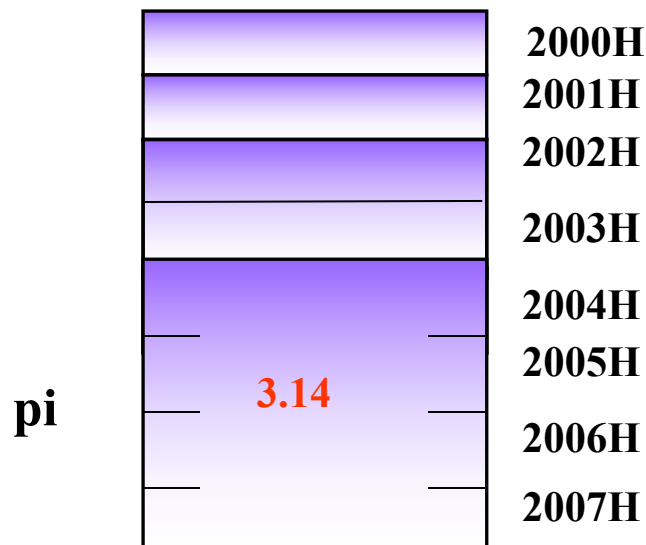
常量设计1：命名常量

◆不能改写的变量(命名常量)

有些变量一旦定义并初始化,便不允许程序去改变该存储空间中的数据。

C语言中定义一个不能改写的变量存储空间

```
const float pi = 3.14;
```



```
#include <math.h>
```

```
✓ double sin( double arg );
```

功能： 函数返回参数arg的正弦值，arg以弧度表示给出。

```
✓ double cos( double arg );
```

功能： 函数返回参数arg的余弦值，arg以弧度表示给出。

```
✓ double tan( double arg );
```

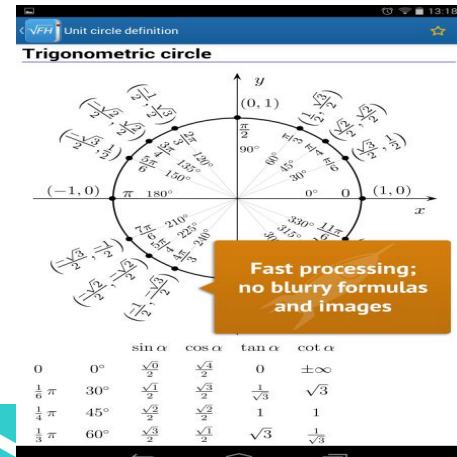
功能： 函数返回参数arg的正切值，arg以弧度表示给出。

```
✓ double acos( double arg );
```

功能： 函数返回参数arg的反余弦值。参数arg应当在-1和1之间。

```
✓ double asin( double arg );
```

```
✓ double atan( double arg );
```



编写求圆柱体表面积的代码

```
#include<stdio.h>
#include<math.h>
int main(void) {
    const double pi = acos(-1.0);
    double r, h, area;
    printf("Enter the radius and height of
cylinder: ");
    scanf("%lf%lf",&r,&h);
    area = (pi*r*r)*2 + (2*pi*r*h);
    printf("Area = %.3f\n", area);
    return 0;
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

测试各种情况，注意条件的边界一定要测，非正常情况也要测试。

测试数据1： 0.5 3.2;

测试数据2： 3.0 4.0;

测试数据3： 2.9 15.2135;

测试数据4： 1.9 0;

- 由于C语言的数据类型严格分为整数和浮点小数两大类型，所以遇到需要将小数转换为整数的情况时，有以下处理方式：
 - 小数四舍五入取整
`(int)(2.5+0.5); (int)(-2.3-0.5);`
 - 小数向上取整
`ceil`函数返回大于等于参数的最小整数
 - 小数向下取整
`floor`函数返回小于等于参数的最大整数

四舍五入取整示例

```
#include <stdio.h>
#include <math.h>
int main(void)
{
    double a=0.0;
    scanf("%lf",&a);
    printf("%f round int is %d\n",a,(int)(a+0.5));
    printf("%f ceil int is %d\n",a,(int)ceil(a));
    printf("%f floor int is %d\n",a,(int)floor(a));

    return 0;
}
```



```
#include <stdio.h>
#include <math.h>
int main(void)
{
    double a=0.0;
    scanf("%lf",&a);
    if (a>0.0)
        printf("%d\n",(int)(a+0.5));
    else
        printf("%d\n",(int)(a-0.5));
    return 0;
}
```

整数除法会舍弃小数

整数除法和浮点数除法不同！浮点数除法的结果是浮点数，而整数除法的结果是整数，没有小数部分。在C语言中，整数除法结果的小数部分被丢弃，这一过程被称为截断（truncation）。

例：计算机一般用秒为单位来计时。给定秒数，换算为小时数、分钟数和剩余秒数，可以用

$(\text{秒数}/3600)=\text{折算小时数}$ ， $\text{剩余数}=(\text{秒数}\%3600)$
再除以60折算分钟数。



秒数折算小时和分钟的代码

```
#include <stdio.h>
#define SEC_PER_MIN 60
#define SEC_PER_HOUR 3600
int main(void)
{
    int seconds, minutes, hours, left;

    printf("Convert seconds to hours and minutes.\n");
    printf("Enter the number of seconds:\n");
    scanf("%d", &seconds);

    hours = seconds / SEC_PER_HOUR;
    minutes = (seconds % SEC_PER_HOUR) / SEC_PER_MIN;
    left = seconds % SEC_PER_MIN;

    printf("%d seconds is %d hours and %d minutes, left %d.\n", seconds, hours, minutes, left);

    return 0;
}
```

符号常量 manifest constant

- 符号常量是仅含有符号名称的值。

C语言定义: **#define** 标识符 替换文本

```
#define TAXRATE 0.015
```

```
owed = TAXRATE * housevalue; .....
```

- 编译时，预处理程序能够将所有出现该符号名称的地方用**值**替换。这称为编译时代入法（compiletime substitution）。
- 使用符号常量比直接使用常量更好！

C程序示例：取余运算

```
/*认识取余运算% */  
#include<stdio.h>  
int main(void){  
    int a,d,result;  
    printf("Please input integers a and d:\n");  
    scanf("%d%d",&a,&d);  
    result = a % d;  
    printf("%d %% %d = %d.\n",a,d,result);  
    return 0;  
}
```

编程活动5. 测试和调试程序

用测试数据运行取余程序，查看结果。

测试数据1: 7, 3;

测试数据2: -7, 3; 负数取余

测试数据3: -3, -4;

测试数据4: 12, 0;

C99规定 “%趋零截断”，如果第1个运算对象是负数，那么取余的结果为负数；如果第1个运算对象是正数，那么取余的结果也是正数。

定义：令 $\mathbf{Z}_m$ 为包含小于 m 的非负整数集：
 $\{0, 1, \dots, m-1\}$

- 运算 $+_m$ 定义为 $a +_m b = (a + b) \bmod m$. 称为模 m 加法.
- 运算 $\cdot_m$ 定义为 $a \cdot_m b = (a \cdot b) \bmod m$. 称为模 m 乘法.
- 使用这些运算就被称为模 m 算术.

例：计算 $7 +_{11} 9$ 和 $7 \cdot_{11} 9$.

解：根据模 m 运算定义可推出：

$$7 +_{11} 9 = (7 + 9) \bmod 11 = 16 \bmod 11 = 5$$

$$7 \cdot_{11} 9 = (7 \cdot 9) \bmod 11 = 63 \bmod 11 = 8$$

取余运算的本质是计算两个整数不能整除的余数。

整数127的表示方法是三位数码：100的倍数+10的倍数+10的余数，因此整数的数位拆解可以用取余运算。

1. 定义程序目标

问题：输入一个三位数，分离出它的百位、十位和个位，反转顺序后输出。

样例1输入： 127

样例1输出： 721

样例2输入： 320

样例2输出： 23

三位数分解的算法设计

三位数分解算法设计：

Begin

定义一个整数变量 n ；

从键盘接受输入的一个三位整数，存入 n ；

显示器输出打印表达式 $n\%10$ 的值；

显示器输出打印表达式 $n/10$ 的值？

显示器输出打印表达式 $n/100$ 的值；

End

算法测试：127。

```
/*三位数反转*/  
#include <stdio.h>  
int main(void)  
{  
    int n;  
  
    printf("Please input a integer  
100~999:\n");  
    scanf("%d",&n);  
    printf("%d%d%d\n",n%10,n/10%10,n/100);  
  
    return 0;  
}
```

4. 编译

目的：检查程序有效性（符合编程语言语法）。

5. 测试和调试程序

用测试数据运行程序，查看结果是否有错。

测试数据1: 127;

测试数据2: 919;

测试数据3: 520;

测试数据4: 300;

测试数据5: -123;

三位数分解算法改进设计：

Begin

定义一个整数变量n和reverse;

从键盘接受输入的一个三位整数，存入n;

$\text{reverse} = (n \% 10) * 100 + (n / 10 \% 10) * 10 + n / 100;$

显示器输出打印reverse的值;

End

算法测试：120，100，101，305，-123。

计算机中，每样事物都是数字，皆可计算。

字符型常量和变量所允许的运算包括：

算术运算： 'b' - 32; //表达式结果为'B'

ch = 'a' + 3; //字母a转换为字母表右边第三个字母

关系运算（按照**ASCII**编码值大小比较）：

if (ch1 == ch2) //字符变量相等

if (ch >= 'A' && ch <= 'Z') //ch值是大写A~Z字母

如何对字母作凯撒加密？

问题：letter+3, 对于字母表末尾的字母会越界！

解决思路：模运算适合表达有限范围的整数运算。

分析：‘X’ +3 如何映射到 ‘A’ ？

字母表内部序号0-25，在模26上加运算可以完成：

$$(23 + 3) \% 26 = 26 \% 26 = 0.$$

算法：只要将字母先转成字母表序号，模加后再从字母表序号转回字母就达到目标。

表达式实现：大写字母转序号 $(\text{letter} - \text{'A'})$ ；

$$((\text{letter} - \text{'A'}) + 3) \% 26 + \text{'A'};$$

字符输入函数getchar()

scanf函数可以接受字符输入，但不如专门的字符输入函数简单易用。

例： `scanf("%c%c",&chA,&chB);`

- ✓ `getchar()` 函数从键盘(STDIN标准输入)获取并返回下一个字符, 如果失败则返回-1。

例： `ch = getchar();`

- ✓ `putchar()` 函数是把一个字符输出到显示器(STDOUT标准输出)。 例： `putchar(ch);`


```
#include<stdio.h>
int main(void){
    char ch,secret;
    printf("Enter a capital letter:");
    ch = getchar();
    secret=((ch- 'A' )+3)%26 + 'A' ;
    putchar(secret);
    return 0;
}
```

